SNAPASSET AI

AI-Assisted IT Asset Label Scanner for Excel Asset Registers

Product Requirements Document (PRD)

&

Software Requirements Specification (SRS)

Android Mobile Application (React Native)

Prepared for: Rajeev (IT Asset Engineer)

Version 1.0 | Draft for Development Planning | July 2026

Document Control

Field	Detail
Document Title	SnapAsset AI - SRS & PRD
Version	1.0
Status	Draft - for review
Platform	Android (native mobile application)
Author	Prepared by Claude (Anthropic) with inputs from Rajeev
Source Data Sample	IT_Asset_PAV_Rajeev_updated.xlsx (multi-store IT asset register)
Sample Label Images	Router.jpeg (Cisco Meraki MX64), Resto_work.jpeg (Restroworks POS unit)

Table of Contents

PART A — PRODUCT REQUIREMENTS DOCUMENT (PRD)

A.1 Problem Statement

Rajeev (and other field IT engineers) manually audit IT and POS hardware assets across multiple restaurant stores (e.g. S507, S576, S375, S103). Each store's asset register is maintained as a tab inside a shared Excel workbook, with columns such as Asset Type, Make, Model, Serial Number, MAC Address, IP Address, and Barcode status. Today, when an engineer physically visits a store, they must:

- Photograph the label on each device (router, POS terminal, printer, monitor, cash drawer, etc.).
- Manually read tiny, often reflective or blurred text for Model No. and Serial No.
- Manually locate the correct row for that asset in the correct store's Excel sheet.
- Manually type the Model No. and Serial No. into the correct cells.

This is slow, error-prone (mistyped serial numbers), and repeated hundreds of times across stores. There is no assistive tool that reads the label directly from a photo and proposes the correct spreadsheet update.

A.2 Product Vision

SnapAsset AI is an Android app that turns a phone camera into a smart asset-entry assistant. The engineer opens the store's Excel workbook inside the app once, then simply photographs (or picks from gallery) each device's label. The app behaves like Google Lens combined with a UPI/QR-style scanner: it detects barcodes/QR codes AND reads printed text (OCR) in the same scan, extracts the Model No. and Serial No., automatically finds the matching row in the already-open sheet (e.g. the row where Asset Type/Make matches "Restroworks" / "Router"), shows the proposed update in an assistant-style side panel, and asks for the engineer's explicit permission before writing anything into the sheet.

A.3 Goals & Objectives

- Reduce time to update one asset record from ~60-90 seconds (manual typing) to under 15 seconds (scan + confirm).
- Eliminate manual typos in Model No. / Serial No. by reading them directly from the label.
- Keep the human in control: no data is written to the Excel file without explicit user confirmation.
- Work across the existing multi-sheet, multi-store workbook structure without changing its layout.
- Work fully offline/on-device where possible, since asset and network data (IP, MAC) can be sensitive.

A.4 Target Users / Persona

Attribute	Description
Persona	Field IT Asset Engineer (e.g. Rajeev)
Context of use	Walking store to store, phone in hand, scanning physical device labels
Technical comfort	Comfortable with Excel and Android apps; not a developer
Primary device	Mid-range Android phone with a working rear camera
Connectivity	Store Wi-Fi may be weak/absent; app must tolerate offline use
Motivation	Finish the audit checklist for a store quickly and accurately, then move to the next store

A.5 Core User Story (as described by the requester)

- Step 1 - I open the app and choose the Excel workbook / the correct store sheet from my drive.
- Step 2 - The app shows me that sheet's data (rows) in an assistant-style side panel.
- Step 3 - I scan a device label (camera live-scan, or pick an existing photo from the gallery / barcode).
- Step 4 - The app behaves like Google Lens / a UPI QR scanner - it reads any barcode/QR AND the printed Model No. / Serial No. text.
- Step 5 - The app automatically finds which row this device belongs to in the already-open sheet (e.g. finds the "Restroworks" / router row) and fills Model No. and Serial No. into an editable preview - it does NOT touch the real file yet.
- Step 6 - The app asks for my permission ("Save this update? Yes / No / Edit") before writing to the sheet.
- Step 7 - Only after I confirm, the app writes the values into the Excel file and shows a saved confirmation.
- Step 8 - I repeat Steps 3-7 for the next device, without re-choosing the sheet each time.

A.6 Scope

A.6.1 In Scope (MVP - Version 1.0)

- Import/open .xlsx workbook from device storage or Google Drive.
- Sheet (tab) picker - select the active store sheet (e.g. S507, S576).
- Read-only preview of the selected sheet's rows in an assistant/side-panel list.
- Camera capture and gallery-image selection for label scanning.
- On-device barcode / QR / Code-128 / Code-39 detection.
- On-device OCR text extraction from the label.
- Field classification: distinguish Model No. vs Serial No. vs MAC vs generic text from the OCR output.
- Auto row-matching engine: match the scanned label to the correct existing row (by Asset Type / Make keyword, e.g. "Router", "Restroworks").
- Assistant panel showing the matched row + proposed Model No./Serial No. before saving.
- Explicit permission dialog before any write to the Excel file.
- Write-back to the correct cells of the correct sheet, preserving all existing formatting/formulas.
- Manual correction of any auto-filled field before confirming.
- Local change history / audit log (what was scanned, what was written, when).
- Fully offline scan + review flow (write-back also works fully offline against the local copy).

A.6.2 Out of Scope (Version 1.0)

- Creating brand-new rows for assets that don't already exist in the sheet (V1 only updates existing rows).
- Multi-user real-time collaborative editing of the same workbook.
- iOS version.
- Automatic upload/sync to a central server or database (V1 is local-file based; cloud sync is a Phase 2 item).
- Editing columns other than Model, Serial Number, and (optionally) MAC/Host fields that are visibly present on the label.

A.7 Key Features Summary

#	Feature	Priority
1	Excel import & store-sheet selector	Must Have
2	Assistant-style sheet preview panel	Must Have
3	Camera + gallery based label capture	Must Have
4	Combined barcode/QR + OCR scan engine	Must Have
5	Auto field classification (Model No. vs Serial No.)	Must Have
6	Auto row-matching to the correct asset row	Must Have
7	Permission/confirmation gate before save	Must Have
8	Write-back into original .xlsx with formatting preserved	Must Have
9	Manual edit/override of detected values	Must Have
10	Continuous/batch scan mode (scan many labels in a row)	Should Have
11	Local audit log / change history	Should Have
12	Duplicate serial number conflict warning	Should Have
13	Offline-first operation	Should Have
14	Google Drive direct open/save	Could Have
15	New-row creation for unlisted assets	Could Have (Phase 2)
16	Cloud sync / central asset database	Won't Have (V1)

A.8 Success Metrics

- SPEED (critical, hard target): a full store audit (approx. 15-20 scannable assets) completed end-to-end - open sheet, scan every device, confirm, save - within 3 minutes total.
- Per-scan AI processing (capture -> proposed Model/Serial/matched row): under 300-400 milliseconds, i.e. feels instant to the user.
- Per-asset full cycle (point camera -> auto-captured -> auto-matched -> confirmed): target 6-10 seconds including the human action of moving to the next device.
- OCR field accuracy: 95%+ correct extraction of Model No. / Serial No. on a clear, well-lit label photo.
- Auto row-match accuracy: 90%+ correct row suggested without manual search.
- Zero unintended writes: 100% of saves are preceded by explicit user confirmation or an undoable auto-confirm window (verified by audit log).
- Adoption: engineer completes a full store's asset list without switching to typing on the Excel app, and without the app ever feeling like the bottleneck.

A.9 Assumptions & Constraints

- The Excel workbook keeps the same column structure as the sample provided (Store Code, Store Name, Asset Type, Make, Model, Serial Number, etc.), one tab per store.

- Labels are physical stickers with reasonably legible printed text and/or a barcode, photographed at a readable distance.
- The app runs on Android 9.0 (API 28) or later, with a functioning camera.
- The workbook file is accessible locally or via a connected cloud-storage app (Drive) that exposes a file the OS file-picker can open.
- Because IP/MAC/serial data can be sensitive, OCR and barcode processing happen on-device; no image or extracted data leaves the device in V1.

PART B — SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

This part follows the general structure of IEEE 830 / ISO-IEC-IEEE 29148 style SRS documents, adapted for an Android application.

B.1 Introduction

B.1.1 Purpose

This document specifies the functional and non-functional requirements for SnapAsset AI, an Android application that lets a field IT engineer open an existing Excel-based asset register, scan physical device labels using the camera, and update Model No. / Serial No. fields in the correct row after explicit confirmation.

B.1.2 Intended Audience

- Mobile app developer(s) building the Android application.
- QA/testers validating scan accuracy and file-safety.
- Rajeev / IT operations team as the requesting stakeholder and primary user.

B.1.3 Definitions, Acronyms, Abbreviations

Term	Definition
OCR	Optical Character Recognition - extracting printed text from an image
Barcode/QR scan	Decoding a 1D barcode or 2D QR code embedded in the label image
Asset Row	A single row in a store's sheet representing one physical device
Assist Panel	The side/overlay panel showing sheet rows and scan suggestions
Write-back	The act of saving extracted values into the actual .xlsx file
Store Sheet	One tab of the workbook, corresponding to one store code (e.g. S507)
Confidence Score	A 0-100% score the app assigns to how sure it is about a detected value or match

B.1.4 References

- Sample workbook: IT_Asset_PAV_Rajeev_updated.xlsx (provided by requester).
- Sample label images: Router.jpeg (Cisco Meraki MX64 router label), Resto_work.jpeg (Restroworks POS unit label).

B.2 Overall Description

B.2.1 Product Perspective

SnapAsset AI is a standalone, self-contained Android app built with React Native (bare CLI, TypeScript). It does not require a backend server for its MVP - it operates directly on a local or cloud-synced copy of the user's .xlsx file using on-device AI (OCR + barcode) models. It is a new product, not a modification of an existing system.

B.2.2 Product Functions (Summary)

- Open and parse a multi-sheet .xlsx workbook.
- Let the user pick the active store sheet.
- Display sheet rows as an assistant-style list, searchable by Asset Type / Make.

- Capture or import a label photo.
- Run barcode/QR detection and OCR on the photo.
- Classify extracted strings into Model No., Serial No., MAC, IP, Host Name where applicable.
- Match the scan to the correct existing row.
- Show a confirmation card and require explicit user permission before writing.
- Write the confirmed values back into the workbook file, preserving all other content and formatting.

B.2.3 User Classes and Characteristics

User Class	Characteristics	Access Level
Field Engineer (primary)	Non-developer; visits stores; scans and confirms updates	Scan, review, confirm/save, edit own entries
Team Lead / Reviewer (future)	Reviews completed store audits	Read-only + "Team Lead Confirmation" field sign-off

B.2.4 Operating Environment

- OS: Android 9.0 (API 28) and above.
- Hardware: phone with an autofocus rear camera (min 8MP recommended for clear label text).
- Storage: local device storage and/or a connected cloud-drive app exposing files via Android's Storage Access Framework (SAF).
- No mandatory internet connection for core scan-and-save flow.

B.2.5 Design and Implementation Constraints

- Must not alter the existing column order, sheet names, or formatting of the workbook - only specific cell values may change.
- Must never write to the file without a prior explicit "Yes/Confirm" action from the user for that specific record.
- Must handle multi-sheet workbooks (one tab per store) as shown in the sample file.

B.2.6 User Documentation

- In-app first-run walkthrough (3-4 screens) explaining: pick sheet -> scan label -> confirm -> saved.
- In-app help tooltip on the confirmation dialog explaining what will be written and where.

B.3 Data Model (derived from the provided workbook)

The workbook contains one sheet per store (e.g. S507, S576, S375, S103, ...). Each sheet shares the same column structure, shown below. SnapAsset AI's scan engine focuses primarily on the highlighted columns (Model, Serial Number) but is aware of all columns for row-matching context.

Column	Example Value	Used by scan engine?
SR No	1, 2, 3 ...	No (row index only)
Store Code	S507	Yes - context filter
Store Name	Building No. 5	No
Location	Gurugram	No
Engg Name	Rajeev	No (auto-filled from logged-in user)
Asset Type	Router, BO-Printer, POS-01 Printer, TAB	Yes - primary match key
OS / OS Version	Android 10.0	Optional - secondary match
Make	Cisco, Epson, Posiflex, Restroworks	Yes - primary match key
Model	MX64-HW, EPSON M2110	Yes - PRIMARY TARGET FIELD
Serial Number	Q2KN-LTQU-Y3RB	Yes - PRIMARY TARGET FIELD
Host Name	TSPLMUMS001POS1	Optional secondary target
IP Address	172.29.10.08	Optional secondary target
Mac Address	F8:9E:28:82:83:68	Optional secondary target
Ram / HDD	4GB / 500GB	No
Asset Status Working - YES/NO	Working	No (not modified by scan)
Barcode status (Yes/No)	Yes	Optionally auto-set to Yes after a successful barcode scan
Barcode Image shared (Yes/No)	Yes	No
Remark	Sticker NA / New Printer	No
Team Lead Confirmation	Yes	No (separate sign-off step)

B.3.1 Sample Label -> Field Mapping (from provided images)

Label Photo	Detected Text / Code	Maps to Column
Router.jpeg (Cisco Meraki MX64)	Model: MX64-HW	Model
Router.jpeg (Cisco Meraki MX64)	S/N: Q2KN-LTQU-Y3RB	Serial Number
Router.jpeg (Cisco Meraki MX64)	MAC: F8:9E:28:82:83:68	Mac Address
Resto_work.jpeg (Restroworks unit)	Model No.: RW POWER MB	Model
Resto_work.jpeg (Restroworks unit)	Barcode value: FX6620RWPRO0102003152	Serial Number

Note: on the Restroworks label the printed barcode number is the effective serial/unit identifier and the barcode itself can be decoded directly, giving higher confidence than OCR alone.

B.4 System Features / Functional Requirements

Each feature is described using: Description, Functional Requirements (FR-IDs), and Priority.

B.4.1 Workbook Import & Sheet Selection

Allows the user to open the .xlsx workbook and choose which store sheet is active for the session.

ID	Requirement	Priority
FR-1.1	The system shall let the user select an .xlsx file from local storage or a connected cloud-drive app using the Android file picker.	Must
FR-1.2	The system shall parse all sheet (tab) names in the workbook and present them as a selectable list (e.g. S507, S576, S375, S103).	Must
FR-1.3	The system shall load the selected sheet's rows and columns into memory, preserving the original column headers and order.	Must
FR-1.4	The system shall allow switching to a different sheet within the same session without re-importing the file.	Should
FR-1.5	If the workbook cannot be parsed (corrupted, unsupported format), the system shall show a clear error and not proceed.	Must

B.4.2 Assistant-Style Sheet Preview Panel

A side panel that shows the currently loaded sheet's rows, acting as the always-visible 'assistant' context for scanning.

ID	Requirement	Priority
FR-2.1	The system shall display the active sheet's rows in a scrollable list/panel, grouped by Asset Type.	Must
FR-2.2	The system shall allow the user to search/filter rows by Asset Type, Make, or Model text.	Should
FR-2.3	The system shall visually flag rows where Model or Serial Number is currently blank (pending data).	Should
FR-2.4	The system shall highlight, in the panel, the row that a completed scan was matched and written to.	Must

B.4.3 Label Capture (Camera / Gallery)

Lets the user provide a label image either by live camera capture or by picking an existing photo.

ID	Requirement	Priority
FR-3.1	The system shall provide a live camera scan mode with a viewfinder overlay guiding label framing.	Must
FR-3.2	The system shall allow picking an existing image from the device gallery/local drive as scan input.	Must
FR-3.3	The system shall request camera and storage/media runtime permissions only when the relevant action is first used, with a clear explanation of why.	Must

ID	Requirement	Priority
FR-3.4	The system shall support continuous scan mode, returning to the camera view immediately after each confirmed save.	Should

B.4.4 Combined Barcode/QR + OCR Scan Engine

The core AI capability: like Google Lens or a UPI/QR scanner, it detects any barcode/QR code AND reads printed text in the same pass.

ID	Requirement	Priority
FR-4.1	The system shall run on-device barcode/QR detection (1D formats such as Code-128/Code-39, and 2D QR) on the captured image.	Must
FR-4.2	The system shall run on-device OCR text recognition on the same captured image, independent of whether a barcode was found.	Must
FR-4.3	The system shall combine barcode and OCR results into a single set of candidate values with a confidence score each.	Must
FR-4.4	The system shall complete scan processing (capture to candidate results) in under 3 seconds on a mid-range device for a single image.	Should
FR-4.5	If neither a barcode nor readable text is detected, the system shall prompt the user to retake the photo with guidance (better lighting/angle/distance).	Must

B.4.5 Field Classification (Model No. vs Serial No. vs Others)

Interprets raw OCR/barcode strings and classifies them into the correct target field, using label layout cues (e.g. text following "Model No.", "S/N", "Serial") and format patterns.

ID	Requirement	Priority
FR-5.1	The system shall detect label keywords ("Model", "Model No.", "S/N", "Serial", "SN", "MAC") and associate the adjacent text as the corresponding value.	Must
FR-5.2	Where no keyword is present, the system shall apply pattern rules (e.g. alphanumeric barcode-linked string = Serial Number; short code near brand/product name = Model) to propose a classification.	Should
FR-5.3	The system shall assign a confidence score to each classified field and only auto-fill fields above a minimum confidence threshold (configurable, default 70%).	Must
FR-5.4	Fields below the confidence threshold shall still be shown to the user, but marked as 'low confidence - please verify' rather than silently auto-filled.	Must

B.4.6 Auto Row-Matching Engine

Determines which existing row in the active sheet the scanned device belongs to (e.g. recognizing 'Restroworks' or 'Cisco/Router' and locating that Asset Type/Make row).

ID	Requirement	Priority
FR-6.1	The system shall match OCR-detected brand/product keywords (e.g. 'Cisco', 'Meraki', 'Restroworks') against the Make and Asset Type columns of the active sheet.	Must

ID	Requirement	Priority
FR-6.2	The system shall present the single best-matching row when confidence is high, or a short ranked list of candidate rows when ambiguous.	Must
FR-6.3	The system shall allow the user to manually pick a different row from the panel if the auto-match is wrong.	Must
FR-6.4	If Model/Serial Number cells in the matched row are already filled, the system shall clearly show the existing value alongside the new scanned value before allowing overwrite.	Must

B.4.7 Review & Permission Gate (Consent Before Save)

No value is ever written to the Excel file without the user explicitly confirming it for that specific record.

ID	Requirement	Priority
FR-7.1	The system shall display a confirmation card showing: matched row (Store/Asset Type/Make), proposed Model No., proposed Serial No., and confidence scores, before any write.	Must
FR-7.2	The system shall provide three explicit actions on the confirmation card: 'Save', 'Edit', and 'Discard'.	Must
FR-7.3	The system shall allow inline editing of any proposed field before confirming Save.	Must
FR-7.4	The system shall NOT modify the underlying .xlsx file in any way until the user taps 'Save'.	Must
FR-7.5	If the user taps 'Discard', the system shall clear the pending scan without touching the file.	Must

B.4.8 Write-Back to Excel

Commits confirmed values into the correct cells of the correct sheet in the real .xlsx file.

ID	Requirement	Priority
FR-8.1	The system shall write only the confirmed cell values (e.g. Model, Serial Number) into the matched row, leaving all other cells and sheets untouched.	Must
FR-8.2	The system shall preserve existing cell formatting, column widths, formulas, and sheet order of the workbook after writing.	Must
FR-8.3	The system shall show a success confirmation (e.g. toast/snackbar: 'Saved: S507 - Router - Model & Serial updated') after a successful write.	Must
FR-8.4	If the write fails (e.g. file locked, storage permission revoked), the system shall show a clear error and keep the pending values available for retry.	Must
FR-8.5	The system shall optionally set 'Barcode status (Yes/No)' to Yes automatically when a save was sourced from a successful barcode/QR decode, subject to the same permission gate.	Could

B.4.9 High-Speed Batch / Continuous Scan Engine (Critical)

The requester's hard requirement: a full store (~15-20 assets) must be scanned and saved in under 3 minutes. This is achieved not by any single trick but by removing every avoidable step and delay from the per-asset loop, described below.

ID	Requirement	Priority
FR-9.1	Auto-capture: the system shall run OCR/barcode detection continuously on the live camera preview (streaming, not single-shot) and automatically lock/capture the instant text or a code is detected and stable for ~150ms - no manual shutter tap required.	Must
FR-9.2	After a confirmed (or auto-confirmed) save, the system shall return directly to the live scan view with the same sheet still active - zero navigation steps between assets.	Must
FR-9.3	Fast-confirm mode: for matches at or above a high-confidence threshold (default 90%), the system shall show a brief (1.5-2 second) auto-save countdown toast ('Saving Router - MX64-HW... tap to cancel/edit') instead of a blocking dialog, so the engineer can keep moving to the next device while low-risk saves complete themselves. Lower-confidence matches always fall back to the full manual confirmation card (FR-7.1-7.4).	Must
FR-9.4	The system shall pre-warm and keep the OCR/barcode ML models resident in memory for the whole session (loaded once at sheet-open), eliminating per-scan model cold-start latency.	Must
FR-9.5	The system shall run barcode detection and OCR in parallel threads on the same frame (not sequentially) to minimize per-frame processing time.	Must
FR-9.6	The system shall maintain a running session summary (e.g. '6 of 21 assets updated for S507 - 1m 42s elapsed') visible in the assist panel at all times.	Must
FR-9.7	Deferred batch file-write: the system shall NOT reopen/re-serialize the .xlsx file after every single confirmed scan (this file I/O, not the AI, is the real bottleneck). Instead it shall hold confirmed values in an in-memory queue and flush them to disk in one fast batch write every few scans and automatically at the end of the session, while still logging each confirmation to the audit trail immediately.	Must
FR-9.8	The system shall pre-fetch/highlight the next likely empty/pending row in the assist panel while the current confirmation is still on screen, so row-matching for the next scan has a head start.	Should
FR-9.9	The system shall allow queuing multiple scans and reviewing/confirming several together in a single batch review screen for engineers who prefer scan-first-confirm-later.	Could

B.4.10 Audit Log / Change History

Keeps a local record of what was scanned, matched, and saved, for accountability and troubleshooting.

ID	Requirement	Priority
FR-10.1	The system shall log, per confirmed save: timestamp, store/sheet, matched row, old value, new value, and source (barcode/OCR/manual).	Should
FR-10.2	The system shall let the user view and export this log (e.g. as CSV) per session or per store.	Could

B.5 End-to-End Workflow (Step by Step)

Step	Actor	Action	System Response
1	User	Opens the app and taps 'Open Workbook'	System shows Android file picker (local + connected drives)
2	User	Selects the .xlsx file	System parses workbook, lists sheet/tab names (S507, S576, ...)
3	User	Selects the active store sheet	System loads that sheet's rows into the Assist Panel
4	User	Simply points the camera at a device label - no shutter tap needed	System streams live frames through OCR + barcode detection in parallel (models already pre-warmed)
5	System (AI)	Auto-captures the instant text/code is detected and stable (~150ms)	System extracts candidate text/codes with confidence scores in under ~400ms
6	System (AI)	Classifies candidates into Model No. / Serial No. / other fields	System proposes field values
7	System (AI)	Matches the scan to a row in the active sheet (e.g. 'Restroworks' row)	System highlights the matched row in the Assist Panel
8a	System	HIGH confidence (>=90%): shows a 1.5-2s auto-save toast	Auto-saves to the in-memory queue unless user taps Cancel/Edit - no blocking dialog
8b	System	LOWER confidence: shows the full confirmation card	Waits for explicit user decision - no write yet
9	User	(Only for 8b) Reviews, edits if needed, taps 'Save'	System adds the confirmed values to the in-memory update queue
10	System	Instantly returns to live scan view for the next device	Queue is flushed to the real .xlsx in one fast batch write every few scans and at session end

B.6 External Interface Requirements

B.6.1 User Interface

- Home screen: 'Open Workbook' action + recent files list.
- Sheet Picker screen: list of tabs found in the workbook.
- Main Scan screen: camera viewfinder (primary), with the Assist Panel as a collapsible side/bottom sheet listing current sheet rows.
- Confirmation Card: bottom-sheet dialog showing matched row summary, proposed Model/Serial fields (editable), Save/Edit/Discard buttons.
- History screen: list of past scans/saves for the current session or store.

B.6.2 Hardware Interfaces

- Rear camera (autofocus) for live label scanning.

- Device flash (optional, user-toggleable) for low-light label scans.

B.6.3 Software Interfaces

- Android Storage Access Framework (SAF), accessed via a React Native document-picker/file-system bridge, for opening/saving files across local storage and cloud-drive apps.
- On-device ML/OCR framework (React Native ML Kit wrappers over Google ML Kit's Text Recognition and Barcode Scanning APIs) for text and barcode extraction.
- Spreadsheet read/write library (exceljs, a JavaScript library) for parsing and writing .xlsx files. Note: exceljs preserves formatting well for standard tabular workbooks like this one, but is not as exhaustive as native libraries (e.g. Apache POI) for exotic Excel features - this was validated early in development against the actual workbook (see Section B.9).

B.7 Non-Functional Requirements

Category	Requirement
Performance (critical)	Scan-to-suggestion under 300-400 milliseconds per frame on a mid-range Android device (4GB RAM class); full store audit (15-20 assets) completed in under 3 minutes end-to-end. See B.4.9 for how this is engineered.
Accuracy	95%+ correct field extraction on clear, well-lit labels; auto-fill only above the configured confidence threshold.
Reliability	No write to the .xlsx file may occur without a prior explicit user confirmation for that record; app must not corrupt the workbook on crash or interruption mid-write (atomic save / write-to-temp-then-replace pattern).
Security & Privacy	Image processing and field extraction happen on-device by default; no asset data (serials, IPs, MACs) is transmitted off-device without explicit opt-in for a future cloud-sync feature.
Usability	A first-time user should be able to complete one full scan-to-save cycle within their first 2 minutes of using the app, guided by the first-run walkthrough.
Offline capability	The entire scan -> match -> confirm -> save flow must work with no network connection.
Compatibility	Android 9.0 (API 28) and above; phones with a working rear autofocus camera.
Data integrity	Existing cell formatting, formulas, sheet order, and unrelated data in the workbook must remain unchanged after any save.
Auditability	Every write is logged locally with a timestamp, source, and before/after value for traceability.

Engineering note on speed: a camera-based AI pipeline cannot meaningfully operate at true nanosecond/microsecond scale - a phone's camera sensor alone takes roughly 10-30 milliseconds just to read one frame, and any on-device ML inference adds tens to a few hundred milliseconds on top. What the app CAN and MUST do is make every step feel instantaneous to the user (sub-400ms per scan) and, more importantly, remove every avoidable step from the workflow (manual shutter taps, per-scan file re-saves, screen navigation) so the total time for a whole store lands under the 3-minute target. Section B.4.9 above lists exactly which techniques deliver this.

B.8 Error Handling & Edge Cases

- Blurred/unreadable label: system asks user to retake photo, offers manual entry as fallback.
- No matching row found in the active sheet: system offers to search other loaded sheets, or flags for manual row selection / (Phase 2) new-row creation.

- Multiple equally-likely matching rows: system shows a ranked candidate list for the user to pick from, rather than guessing.
- Serial Number already present and different from the scanned value: system shows a conflict warning ('Existing: X, Scanned: Y') and requires explicit confirmation to overwrite.
- File became read-only or was moved/deleted since opening: system detects the write failure and prompts the user to re-select the file, preserving the pending unsaved values.
- Permission (camera/storage) denied: system explains why the permission is needed and offers a retry / settings shortcut, without crashing.

B.9 High-Level Architecture

- UI Layer - React Native (TypeScript) screens with React Navigation: Home, Sheet Picker, Scan, Confirmation, History.
- Capture Layer - react-native-vision-camera for live capture and frame processing; native document/gallery picker for existing images.
- AI/Scan Engine - Barcode/QR decoder + OCR text recognizer running on-device, producing raw candidate strings with bounding boxes and confidence.
- Field Classification & Matching Engine - Rule/keyword-based classifier (extensible to a lightweight on-device model later) that labels candidates as Model/Serial/etc. and matches them to a sheet row.
- Consent/Permission Layer - Enforces that no write proceeds without explicit user confirmation of the specific proposed change.
- Excel I/O Engine - Reads and writes .xlsx via exceljs, operating on the specific matched cells only, using a safe write-then-replace pattern to avoid file corruption.
- Local Storage - Session state, recent files list, and the audit/change-history log (e.g. Room database).

B.10 Suggested Technology Stack

Selected to match the assigned developer's existing skillset (React, Node.js/MERN, Docker/Kubernetes, GitHub Actions CI/CD) so implementation can begin without a new language/platform learning curve.

Layer	Suggested Technology
Framework	React Native (bare CLI, not Expo Go - required for custom native camera/ML modules)
Language	TypeScript
UI / Navigation	React Native components + React Navigation
State management	Redux Toolkit
Camera capture	react-native-vision-camera (with frame processors)
Barcode/QR scanning	@react-native-ml-kit/barcode-scanning (on-device)
OCR	@react-native-ml-kit/text-recognition (on-device)
Excel read/write	exceljs (validated against the real workbook for formatting fidelity before feature build-out)
File access	react-native-document-picker + react-native-fs, backed by Android's Storage Access Framework (SAF)
Local persistence	react-native-sqlite-storage or MMKV (for history/audit log and session state)

Layer	Suggested Technology
Architecture pattern	Container/presentational components with a dedicated matching-engine module, kept as pure/testable TypeScript functions
CI/CD	GitHub Actions - lint, TypeScript check, Jest unit tests, and automated debug-APK build on every push

B.11 Future Enhancements (Phase 2+)

- Allow creating a brand-new asset row when a scanned device has no existing match.
- Cloud sync of the workbook and audit logs across engineers/devices.
- Team Lead review/sign-off workflow directly in-app (mapping to the 'Team Lead Confirmation' column).
- Smarter AI matching using a small on-device model trained on the organization's own label formats for higher accuracy across varied device brands.
- Voice-guided hands-free scanning for engineers working with both hands occupied.

B.12 Acceptance Criteria (MVP Definition of Done)

- User can open the provided sample workbook and see all its store sheets listed.
- User can select a sheet and see its rows in the Assist Panel.
- User can scan the provided Router.jpeg-style label and the app proposes Model = MX64-HW, Serial = Q2KN-LTQU-Y3RB, matched to the Router row.
- User can scan the provided Resto_work.jpeg-style label and the app proposes Model = RW POWER MB and the barcode-derived serial, matched to the appropriate row.
- No cell in the workbook changes until the user explicitly taps Save on the confirmation card.
- After Save, opening the workbook in Excel/Google Sheets shows the correct values in the correct cells, with everything else unchanged.
- A test run scanning all scannable assets of one store sheet (~15-20 devices) completes, including the final file save, in under 3 minutes on a mid-range test device.

End of Document